

Modul Praktikum Flutter Fundamental

Pertemuan 5: Penyimpanan Lokal (SharedPreferences dan SQLite)

Durasi	Level	Prasyarat
150 menit	Menengah	Pertemuan 1–4 (form, navigasi, async, FutureBuilder)

1. Tujuan Pembelajaran

Setelah praktikum ini, mahasiswa mampu:

1. Memilih jenis penyimpanan lokal yang tepat: key-value, database, atau berkas.
2. Menyimpan dan membaca pengaturan sederhana dengan `shared_preferences`.
3. Merancang tabel dan membuat database SQLite dengan `sqflite`.
4. Mengimplementasikan operasi CRUD (*Create, Read, Update, Delete*) dan menghubungkannya dengan UI.
5. Memastikan data tetap ada setelah aplikasi ditutup dan dibuka kembali.

2. Alat dan Bahan

- Flutter SDK, editor, dan **emulator/perangkat Android** (sqflite paling stabil di Android/iOS)
- Proyek baru: `flutter create praktikum_5`
- Koneksi internet untuk mengunduh paket (setelah itu tidak diperlukan)

3. Teori Singkat

Data di memori (variabel, `State`) hilang begitu aplikasi ditutup. Agar data bertahan, aplikasi menyimpannya di penyimpanan perangkat (*persistence*).

Pilihan	Cocok untuk	Paket
Key-value	Pengaturan kecil: tema, nama, status login, angka terakhir	<code>shared_preferences</code>
Database SQLite	Data terstruktur dan banyak: catatan, transaksi, daftar produk	<code>sqflite</code> , <code>path</code>
Berkas	Dokumen, gambar, ekspor/impor data	<code>path_provider</code> , <code>dart:io</code>

CRUD dan SQL. Setiap operasi CRUD memiliki padanan perintah SQL:

CRUD	SQL	Method sqflite
Create	INSERT	<code>db.insert()</code>
Read	SELECT	<code>db.query()</code>
Update	UPDATE	<code>db.update()</code>
Delete	DELETE	<code>db.delete()</code>

Aman dari SQL injection: jangan menyusun query dengan menyambung teks dari pengguna. Gunakan tanda tanya sebagai *placeholder* lewat `where: 'id = ?'`, `whereArgs: [id]`.

4. Langkah Praktikum

Bagian A: SharedPreferences (Pengaturan Sederhana)

Pasang paket, lalu buat aplikasi yang menyimpan nama, pilihan mode gelap, dan hitungan berapa kali aplikasi dibuka:

```
flutter pub add shared_preferences

import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Praktikum 5',
      home: const PengaturanPage(),
    );
  }
}

class PengaturanPage extends StatefulWidget {
  const PengaturanPage({super.key});

  @override
  State<PengaturanPage> createState() => _PengaturanPageState();
}

class _PengaturanPageState extends State<PengaturanPage> {
  final _controller = TextEditingController();
  bool _gelap = false;
  int _bukaKe = 0;

  @override
  void initState() {
    super.initState();
    _muat();
  }

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  Future<void> _muat() async {
    final prefs = await SharedPreferences.getInstance();
    if (!mounted) return;
    setState(() {
      _controller.text = prefs.getString('nama') ?? '';
      _gelap = prefs.getBool('gelap') ?? false;
      _bukaKe = (prefs.getInt('bukaKe') ?? 0) + 1;
    });
    await prefs.setInt('bukaKe', _bukaKe);
  }

  Future<void> _simpanNama() async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setString('nama', _controller.text.trim());
    if (!mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Nama disimpan')),
    );
  }

  Future<void> _ubahTema(bool nilai) async {
    setState(() => _gelap = nilai);
  }
}
```

```

    final prefs = await SharedPreferences.getInstance();
    await prefs.setBool('gelap', nilai);
  }

  @override
  Widget build(BuildContext context) {
    return Theme(
      data: ThemeData(
        brightness: _gelap ? Brightness.dark : Brightness.light,
        colorSchemeSeed: Colors.blue,
        useMaterial3: true,
      ),
      child: Scaffold(
        appBar: AppBar(title: const Text('Pengaturan')),
        body: Padding(
          padding: const EdgeInsets.all(16),
          child: Column(
            children: [
              TextField(
                controller: _controller,
                decoration: const InputDecoration(
                  labelText: 'Nama',
                  border: OutlineInputBorder(),
                ),
              ),
              const SizedBox(height: 12),
              ElevatedButton(
                onPressed: _simpanNama,
                child: const Text('Simpan nama'),
              ),
              SwitchListTile(
                title: const Text('Mode gelap'),
                value: _gelap,
                onChanged: _ubahTema,
              ),
              const SizedBox(height: 12),
              Text('Aplikasi dibuka ke-$_bukaKe kali'),
            ],
          ),
        ),
      ),
    );
  }
}

```

Checkpoint: isi nama, aktifkan mode gelap, lalu **hentikan aplikasi sepenuhnya** (Stop, bukan hot reload) dan jalankan lagi. Nama dan mode gelap harus tetap ada, dan hitungan bertambah setiap kali dibuka.

Bagian B: Menyiapkan SQLite

Buat proyek baru (atau lanjutkan di proyek yang sama), lalu pasang paket:

```
flutter pub add sqflite path
```

Buat model Catatan dan kelas pembantu database DBHelper. Letakkan di main.dart (atau berkas terpisah), dengan import berikut di atas:

```

import 'package:path/path.dart' as p;
import 'package:sqflite/sqflite.dart';

class Catatan {
  final int? id;
  final String judul;
  final String isi;

  const Catatan({this.id, required this.judul, required this.isi});

  Map<String, Object?> toMap() => {'id': id, 'judul': judul, 'isi': isi};
}

```

```

    factory Catatan.fromMap(Map<String, Object?> m) => Catatan(
        id: m['id'] as int,
        judul: m['judul'] as String,
        isi: m['isi'] as String,
    );
}

class DbHelper {
    static Database? _db;

    static Future<Database> get database async {
        if (_db != null) return _db!;
        final path = p.join(await getDatabasesPath(), 'catatan.db');
        _db = await openDatabase(
            path,
            version: 1,
            onCreate: (db, version) {
                return db.execute(
                    'CREATE TABLE catatan('
                    'id INTEGER PRIMARY KEY AUTOINCREMENT, '
                    'judul TEXT NOT NULL, '
                    'isi TEXT NOT NULL)',
                );
            },
        );
        return _db!;
    }

    static Future<int> tambah(Catatan c) async {
        final db = await database;
        return db.insert('catatan', c.toMap());
    }

    static Future<List<Catatan>> semua() async {
        final db = await database;
        final rows = await db.query('catatan', orderBy: 'id DESC');
        return rows.map(Catatan.fromMap).toList();
    }

    static Future<int> ubah(Catatan c) async {
        final db = await database;
        return db.update('catatan', c.toMap(), where: 'id = ?', whereArgs: [c.id]);
    }

    static Future<int> hapus(int id) async {
        final db = await database;
        return db.delete('catatan', where: 'id = ?', whereArgs: [id]);
    }
}

```

Checkpoint: kode tidak ada galat kompilasi. Perhatikan: id bernilai null saat catatan baru dibuat, lalu diisi otomatis oleh SQLite (AUTOINCREMENT).

Bagian C: Halaman Daftar Catatan (Read dan Delete)

Ubah home : pada MyApp menjadi const CatatanPage(), lalu tambahkan:

```

class CatatanPage extends StatefulWidget {
    const CatatanPage({super.key});

    @override
    State<CatatanPage> createState() => _CatatanPageState();
}

class _CatatanPageState extends State<CatatanPage> {
    late Future<List<Catatan>> _future;

    @override
    void initState() {
        super.initState();
    }
}

```

```

    _future = DbHelper.semua();
  }

  void _muat() {
    setState(() {
      _future = DbHelper.semua();
    });
  }

  Future<void> _buka([Catatan? catatan]) async {
    await Navigator.push(
      context,
      MaterialPageRoute(builder: (_) => FormCatatanPage(catatan: catatan)),
    );
    if (!mounted) return;
    _muat();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Catatan Saya')),
      body: FutureBuilder<List<Catatan>>({
        future: _future,
        builder: (context, snapshot) {
          if (snapshot.connectionState == ConnectionState.waiting) {
            return const Center(child: CircularProgressIndicator());
          }
          if (snapshot.hasError) {
            return Center(child: Text('Galat: ${snapshot.error}'));
          }
          final data = snapshot.data!;
          if (data.isEmpty) {
            return const Center(child: Text('Belum ada catatan'));
          }
          return ListView.builder(
            itemCount: data.length,
            itemBuilder: (context, i) {
              final c = data[i];
              return ListTile(
                title: Text(c.judul),
                subtitle: Text(c.isi, maxLines: 2, overflow: TextOverflow.ellipsis),
                onTap: () => _buka(c),
                trailing: IconButton(
                  icon: const Icon(Icons.delete),
                  onPressed: () async {
                    await DbHelper.hapus(c.id!);
                    _muat();
                  },
                ),
              );
            },
          );
        },
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () => _buka(),
        child: const Icon(Icons.add),
      ),
    );
  }
}

```

Bagian D: Form Tambah dan Ubah Catatan (Create dan Update)

Satu halaman form dipakai untuk dua keperluan: bila `catatan` kosong berarti tambah, bila terisi berarti ubah.

```

class FormCatatanPage extends StatefulWidget {
  final Catatan? catatan;
  const FormCatatanPage({super.key, this.catatan});
}

```

```

@override
State<FormCatatanPage> createState() => _FormCatatanPageState();
}

class _FormCatatanPageState extends State<FormCatatanPage> {
  final _formKey = GlobalKey<FormState>();
  late final TextEditingController _judul;
  late final TextEditingController _isi;

  @override
  void initState() {
    super.initState();
    _judul = TextEditingController(text: widget.catatan?.judul ?? '');
    _isi = TextEditingController(text: widget.catatan?.isi ?? '');
  }

  @override
  void dispose() {
    _judul.dispose();
    _isi.dispose();
    super.dispose();
  }

  Future<void> _simpan() async {
    if (!_formKey.currentState!.validate()) return;
    final c = Catatan(
      id: widget.catatan?.id,
      judul: _judul.text.trim(),
      isi: _isi.text.trim(),
    );
    if (widget.catatan == null) {
      await DbHelper.tambah(c);
    } else {
      await DbHelper.ubah(c);
    }
    if (!mounted) return;
    Navigator.pop(context);
  }

  @override
  Widget build(BuildContext context) {
    final baru = widget.catatan == null;
    return Scaffold(
      appBar: AppBar(title: Text(baru ? 'Catatan Baru' : 'Ubah Catatan')),
      body: Form(
        key: _formKey,
        child: ListView(
          padding: const EdgeInsets.all(16),
          children: [
            TextFormField(
              controller: _judul,
              decoration: const InputDecoration(
                labelText: 'Judul',
                border: OutlineInputBorder(),
              ),
              validator: (v) =>
                (v == null || v.trim().isEmpty) ? 'Judul wajib diisi' : null,
            ),
            const SizedBox(height: 12),
            TextFormField(
              controller: _isi,
              maxLines: 6,
              decoration: const InputDecoration(
                labelText: 'Isi catatan',
                border: OutlineInputBorder(),
              ),
              validator: (v) =>
                (v == null || v.trim().isEmpty) ? 'Isi wajib diisi' : null,
            ),
            const SizedBox(height: 16),
            ElevatedButton(onPressed: _simpan, child: const Text('Simpan')),
          ],
        ),
      ),
    );
  }
}

```

```

    },
    ),
  );
}
}

```

Checkpoint: (1) menambah catatan memunculkannya di daftar; (2) mengetuk catatan membuka form terisi dan menyimpannya memperbarui data; (3) ikon sampah menghapus; (4) **tutup aplikasi sepenuhnya dan buka lagi: semua catatan masih ada.**

5. Latihan Mandiri

1. Tampilkan `AlertDialog` konfirmasi "Hapus catatan ini?" sebelum data dihapus.
2. Tambahkan kolom pencarian yang memfilter daftar memakai `where: 'judul LIKE ?'`, `whereArgs: ['%$kata%']`.
3. Tambahkan kolom `dibuat` (tanggal dibuat). Naikkan `version` menjadi 2 dan tambahkan `onUpgrade` berisi `ALTER TABLE catatan ADD COLUMN dibuat TEXT`. Amati apa yang terjadi bila `version` tidak dinaikkan. Petunjuk: (a) tambahkan juga kolom `dibuat` pada `CREATE TABLE` di `onCreate` agar instalasi baru langsung memiliki kolom tersebut; (b) tambahkan properti `String?` `dibuat` pada `Catatan` beserta `toMap()/fromMap()`. Tipenya `nullable` karena catatan lama bernilai `NULL`.
4. Simpan pilihan urutan (terbaru/terlama) dengan `shared_preferences` dan terapkan pada `orderBy`.

6. Tugas

Buat aplikasi **Pencatat Pengeluaran** dengan ketentuan:

- Tabel `pengeluaran`: `id`, `nama`, `jumlah` (`integer`), `kategori`, dan `tanggal`.
- CRUD lengkap: daftar, tambah, ubah, hapus, dengan form berisi validasi (nama wajib, jumlah angka lebih dari 0, kategori dari dropdown).
- Menampilkan **total pengeluaran** di halaman utama (dihitung di Dart atau memakai `SELECT SUM(jumlah)`).
- Menyimpan satu pengaturan (misalnya mode gelap atau kategori filter terakhir) dengan `shared_preferences`.
- Data harus tetap ada setelah aplikasi ditutup dan dibuka kembali.
- Kumpulkan: tangkapan layar (daftar, form, dan bukti data bertahan setelah restart) serta berkas kode (atau tautan repositori).

7. Rubrik Penilaian

Komponen	Bobot
Bagian A–D berjalan (checkpoint)	30%
Latihan mandiri	20%
Tugas Pencatat Pengeluaran (CRUD, validasi, total, data bertahan)	40%
Kerapian kode (model, akses data, UI terpisah)	10%

8. Pertanyaan Refleksi

1. Mengapa `shared_preferences` kurang cocok untuk menyimpan ratusan catatan? Apa pilihan yang lebih baik?
2. Mengapa query memakai placeholder ? dan `whereArgs`, bukan menyambung teks langsung?
3. Mengapa setelah kembali dari halaman form, daftar perlu dimuat ulang? Bagaimana caranya pada kode ini?
4. Apa fungsi `onCreate` dan `onUpgrade`, dan kapan masing-masing dijalankan?

9. Troubleshooting Umum

Masalah	Solusi
<code>MissingPluginException</code>	Hentikan aplikasi sepenuhnya lalu jalankan ulang (<code>flutter run</code>); hot reload/restart tidak memuat plugin baru
Data baru tidak muncul di daftar	Pastikan <code>_muat()</code> dipanggil setelah tambah/ubah/hapus
Error: no such table / no such column	Skema diubah tanpa menaikkan <code>version</code> . Saat pengembangan, hapus data aplikasi (uninstall atau Clear data) lalu jalankan ulang
Bernilai null pada pembacaan awal	Gunakan nilai bawaan: <code>prefs.getString('x') ?? ''</code>
sqlite tidak jalan di web/Windows/Linux	Gunakan emulator Android, atau paket <code>sqflite_common_ffi</code> untuk desktop
Peringatan <code>use_build_context_synchronously</code>	Periksa <code>if (!mounted) return;</code> setelah setiap <code>await</code> sebelum memakai <code>context</code> atau <code>setState</code>

10. Referensi

- Menyimpan data key-value: docs.flutter.dev/cookbook/persistence/key-value
- Membaca dan menulis SQLite: docs.flutter.dev/cookbook/persistence/sqlite
- Paket `shared_preferences`: pub.dev/packages/shared_preferences
- Paket `sqflite`: pub.dev/packages/sqflite